

Sputnik Devs

Skystride

Game Design Document

Team:

Cătălin Grigoriev – Engine Programmer

Tudor Baciuc – Level Designer

Ilie Beleniuc – 3D Environment Artist

Last update : 19.12.2025

Contents

I	Introduction	5
I.1	Purpose	5
I.2	Scope	5
II	Game Overview	6
II.1	Game Concept	6
II.2	Target Audience	6
II.3	Genre(s)	7
II.4	Game Flow Summary	8
II.4.1	Level Selection / Progression	8
II.4.2	Gameplay Loop	8
II.4.3	Objectives & Goals	9
II.4.4	Level Completion	9
II.4.5	Game Over / Retry	9
II.4.6	Endgame & Rewards	9
II.5	Look and Feel	10
III	Gameplay	11
III.1	Primary Gameplay Objectives	11
III.2	Challenge and Skill Objectives	11
III.3	Exploration and Discovery Objectives	12
III.4	Game Progression	12
III.4.1	Introduction / Tutorial Levels	12
III.4.2	Early Levels – Foundation Building	12
III.4.3	Intermediate Levels – Increasing Complexity	12
III.4.4	Advanced Levels – Mastery and Challenge	12
IV	Mechanics (Key Section)	13
IV.1	Rules	13
IV.1.1	Explicit Rules	13
IV.1.2	Implicit Rules	14
IV.2	Model of the game universe	15
IV.2.1	Core Components of the Universe	15
IV.2.2	Interaction Model	16

IV.2.3	Summary	16
IV.3	Physics.....	17
IV.3.1	Rigid Body Dynamics (unimplemented).....	17
IV.3.2	Gravity and Environmental Forces (implemented).....	17
IV.3.3	Collision Detection and Response (implemented AABB system).....	17
IV.3.4	Player Movement Physics (implemented).....	17
IV.3.5	Object Interaction (unimplemented)	17
IV.4	Economy.....	18
IV.4.1	Resource Collection	18
IV.4.2	Survival Mechanics	18
IV.4.3	Incentivized Exploration	18
IV.5	Objects.....	19
V	Story and Narrative	23
V.1	Current Narrative Status	23
V.2	Current Narrative Context	23
V.3	Potential for Future Narrative Integration	23
VI	Game World	24
VI.1	Maps and Environments	24
VII	Interface.....	26
VII.1	HUD (Heads-Up Display)	26
VII.2	Menus	26
VII.3	Camera Model	27
VIII	Audio, music, sound effects	27
IX	Game Art – intended style.....	27
IX.1	Visual Style and Inspiration	27
IX.2	Characters and Entities	28
IX.3	Objects and Interactive Elements	28
X	Map Editor.....	28
X.1	Editor Overview	28
X.2	Gizmo Controls	29
X.3	Game Synchronization	29
X.4	Additional Features	29
XI	Engine Systems	30

XI.1	Inventory	30
XI.2	Loading Screen.....	30
XI.3	Axis-Aligned Bounding Box Collision	30
XI.4	Sound Manager	30
XI.5	Non-Player Character (NPC)	31
XI.6	Distance Rendering System	31

I Introduction

I.1 Purpose

This Game Design Document (GDD) describes the architecture, system structure, and detailed design of **Skystride**, a three-dimensional platformer game that integrates advanced physics simulation and dynamically interacting entities.

The purpose of this document is to provide a comprehensive technical reference by defining how the game's software components are organized, how they communicate, and how core systems such as physics, rendering, input and game logic are implemented.

I.2 Scope

Skystride is a three-dimensional platformer game designed to provide an immersive and interactive gameplay experience through physics simulation and intelligent, dynamic entities. The software will allow players to navigate complex 3D environments, interact with moving and reactive objects, and solve challenges that are influenced by real-time physical behavior.

The scope of this project includes the development of the core game engine systems such as rendering, physics, collision detection, input handling, artificial intelligence, and gameplay logic. The game will feature dynamic entities such as platforms, enemies, and objects that respond to gravity, forces, and player interactions. These systems will work together to create a responsive and believable virtual world.

The main goals of the project are:

- To create a smooth and engaging 3D environment experience
- To implement advanced physics simulation for realistic movement, collisions, and object interactions
- To support dynamic entities that adapt to the player's actions and environment

The objectives of the software are:

- To provide accurate and stable physical simulations in real time
- To offer intuitive controls and responsive character movement
- To ensure modular and scalable system design for future content and features

The benefits of this project include:

- A flexible architecture that allows new levels, mechanics, and entities to be added easily

II Game Overview

II.1 Game Concept

Skystride is a three-dimensional platformer game that focuses on physics-based interactions and dynamic environments. The player controls a character who must navigate through complex 3D levels filled with moving platforms, interactive objects, obstacles, and intelligent entities.

Unlike traditional platformers, this game emphasizes advanced physical simulation. Every object in the environment responds to gravity, forces, collisions, and player interaction, creating unpredictable and engaging gameplay. Platforms may tilt, slide, collapse, or react to weight, while dynamic entities such as enemies, traps, and movable objects interact naturally with both the player and the environment.

The core gameplay revolves around exploration, precise movement, and problem-solving. Players must use timing, physics, and environmental interactions to overcome challenges, reach new areas, and complete levels. The dynamic nature of the game world ensures that no two playthroughs feel exactly the same.

II.2 Target Audience

The target audience includes:

- **Casual and core gamers** who appreciate exploration, problem-solving, and precision-based platforming.
- **Fans of physics-based games** who enjoy interacting with objects and environments that behave realistically.
- **Students and enthusiasts of game design and physics simulations** who want to study or experience advanced game mechanics in an interactive environment.
- **Age group:** Teenagers and adults (13+) due to moderate complexity and skill-based gameplay.
- **Platforms:** Players using PCs capable of rendering 3D environments and handling physics-intensive simulations.

II.3 Genre(s)

Game falls into the following genres:

- **3D Platformer:** The core gameplay involves navigating a three-dimensional environment, jumping across platforms, climbing, and avoiding obstacles.
- **Physics-Based Game:** Gameplay mechanics are driven by realistic physics simulations, where objects and environments respond naturally to forces, collisions, and interactions.
- **Action-Adventure:** Players explore dynamic levels, solve environmental puzzles, and overcome challenges presented by intelligent entities and moving obstacles.
- **Simulation Elements:** Advanced physics and entity behavior add a layer of simulation, providing emergent gameplay scenarios that vary with each playthrough.

This combination of genres creates an immersive and engaging gameplay experience that emphasizes skillful movement, problem-solving, and interaction with a dynamic 3D world.

II.4 Game Flow Summary

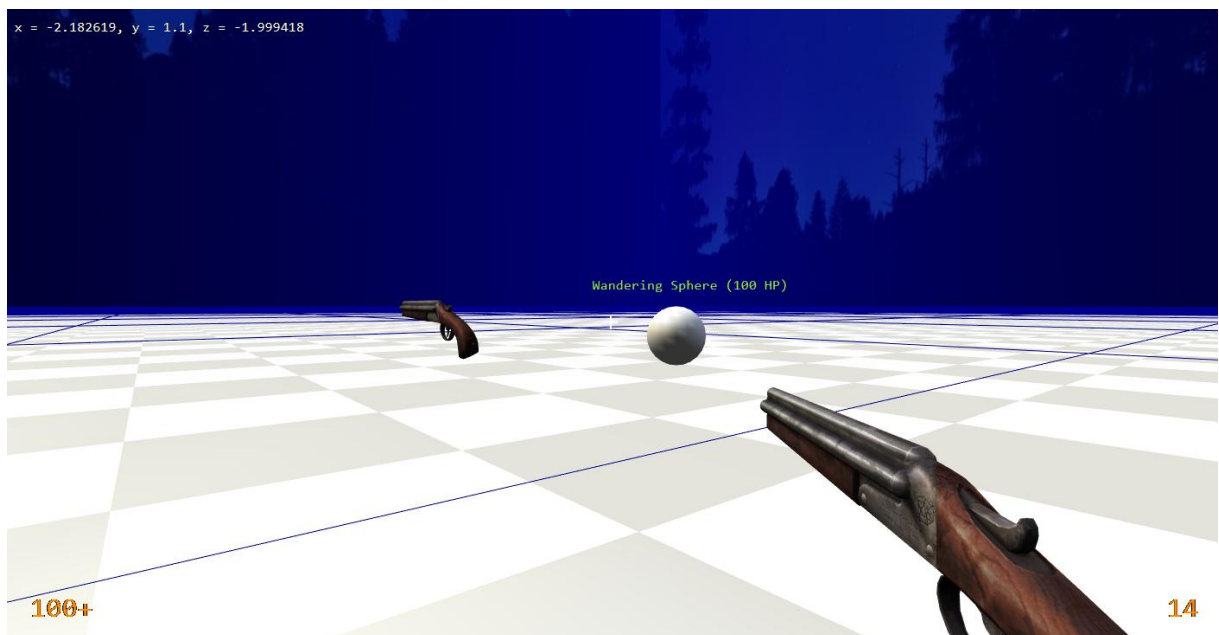
II.4.1 Level Selection / Progression

Players select a level or continue from their last checkpoint. Each level increases in complexity, introducing new dynamic platforms, obstacles, and intelligent entities.



II.4.2 Gameplay Loop

Players navigate an unfamiliar environment populated with adversaries and obstacles. They interact with dynamic elements, including moving platforms, levers, and environmental hazards. Non-player characters (NPCs) and environmental objects respond to player actions, generating emergent and unpredictable scenarios.



II.4.3 Objectives & Goals

Complete platforming challenges, reach checkpoints, and achieve level-specific goals (e.g., collect items, activate mechanisms, defeat enemies).

II.4.4 Level Completion

Upon completing all objectives, the player advances to the next level, with difficulty scaling and new mechanics introduced progressively.

II.4.5 Game Over / Retry

Failure occurs if the player falls into hazards or fails objectives. The player can retry from the last checkpoint without restarting the entire level.

II.4.6 Endgame & Rewards

Completing all levels unlocks additional content, achievements, or bonus challenges, encouraging replayability and mastery of physics-based mechanics.

II.5 Look and Feel

Platform is designed to be visually engaging, immersive, and intuitive, reflecting the dynamic and physics-driven nature of its gameplay. The look and feel of the game can be described as follows:

Visual Style:

- **Dynamic environments:** Platforms, obstacles, and objects visually respond to interactions and physics forces (tilting, sliding, breaking, or bouncing), reinforcing the gameplay mechanics.
- **Color palette:** Bright and contrasting colors are used for key interactive objects and hazards, while the environment uses natural or thematic tones (e.g., forest, industrial, or futuristic levels) to guide the player visually.

Character and Entity Design:

- Main character and dynamic entities have smooth, expressive animations that reflect physics interactions, such as momentum, jumps, or collisions.
- Enemies and NPCs exhibit movement patterns and behaviors that are visually intuitive, allowing the player to predict actions and react accordingly.

Camera and Perspective:

- First-person perspective, providing a clear view of the environment and character interactions.

Overall Feel:

- The game conveys a sense of motion, weight, and responsiveness, giving players feedback through both visual and physical interactions.
- The combination of dynamic physics, environmental reactions, and stylized visuals creates a playful yet challenging atmosphere that encourages experimentation and exploration.

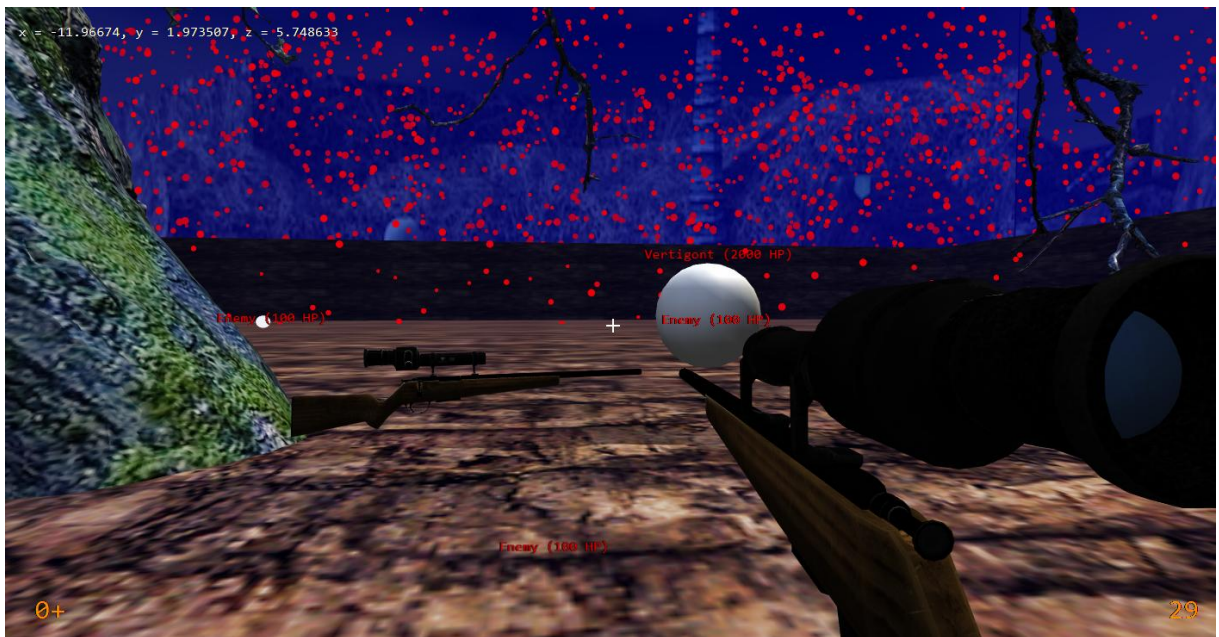
III Gameplay

The primary objectives of the game are structured to offer players engaging challenges that integrate skill, strategic decision-making, and interaction within a dynamic, physics-driven environment. These objectives include:

III.1 Primary Gameplay Objectives

The game challenges players to navigate intricate 3D environments filled with platforms, obstacles, and hazards. Players must complete each level by reaching designated endpoints or achieving specific objectives, such as activating mechanisms, collecting items, or solving environmental puzzles.

Throughout the game, players interact with dynamic entities and objects that respond realistically to physics, including moving platforms, collapsible structures, and interactive crates. **For example, in the “Vertigont” map, players are required to gather ammunition and weapons in order to defeat the primary Sphere Boss.**



III.2 Challenge and Skill Objectives

Players are expected to master precise movement and timing to successfully navigate platforming and physics-based challenges. They must anticipate and adapt to dynamic changes in the environment resulting from physics interactions or the behavior of entities. Additionally, players are encouraged to strategically utilize environmental features and interactive objects to progress through challenging areas.

III.3 Exploration and Discovery Objectives

Players explore visually diverse and interactive 3D environments to discover hidden paths, collectibles, and secrets. They are encouraged to experiment with physics-based interactions to uncover creative solutions to puzzles and overcome challenges.

III.4 Game Progression

The game is designed to deliver a structured yet dynamic progression, with difficulty, complexity, and player engagement increasing gradually. The progression framework is organized as follows:

III.4.1 Introduction / Tutorial Levels

Players are initially introduced to fundamental controls, character movement, and environmental interaction. Simple physics-based objects and platforms provide opportunities to practice jumping, running, climbing, and engaging with dynamic elements.

Tutorial sequences progressively introduce additional mechanics, such as moving platforms, and basic entity interactions.

III.4.2 Early Levels – Foundation Building

Early levels are designed to reinforce movement skills while introducing minor challenges, including small gaps, simple moving platforms, and static obstacles.

Physics interactions are integrated progressively to help players understand cause-and-effect relationships within the game world. Initial enemy encounters and environmental hazards are presented in a controlled manner to ensure a manageable learning curve.

III.4.3 Intermediate Levels – Increasing Complexity

Intermediate levels feature larger areas, increasingly complex platform arrangements, and timed or moving obstacles. Dynamic entities become more interactive and present greater challenges, requiring strategic planning and quick reflexes. Environmental puzzles that combine multiple physics-based elements are introduced to encourage creative problem-solving.

III.4.4 Advanced Levels – Mastery and Challenge

Advanced levels demand mastery of all previously introduced mechanics. Complex combinations of moving platforms, dynamic hazards, and intelligent enemies challenge both reflexes and problem-solving abilities. Emergent gameplay arises as multiple physics-driven elements interact, generating unique challenges in each playthrough.

The game progression is carefully designed to balance learning, challenge, and exploration, ensuring that players are gradually introduced to new mechanics while being consistently engaged by dynamic physics-based interactions and intelligent entities.

IV Mechanics

IV.1 Rules

Game has a set of **explicit** and **implicit** rules that govern gameplay, interactions, and player behavior. These rules ensure a consistent, fair, and engaging experience.

IV.1.1 Explicit Rules

These are clearly defined rules that the player is aware of and must follow to succeed:

1. Movement and Navigation:

- The player can move, jump, climb, and interact with objects according to the game's controls.
- Falling off platforms or into hazards results in failure or loss of progress, requiring the player to retry from the last checkpoint.

2. Objectives and Goals:

- Players must complete level-specific goals, such as reaching the exit, activating mechanisms, collecting items, or defeating certain enemies.
- Progression to the next level is only possible after completing all objectives in the current level.

3. Interaction with Dynamic Objects:

- Players must use physics-based mechanics to manipulate objects, such as pushing crates, activating levers, or riding moving platforms.
- Misuse of objects (e.g., dropping a platform) may hinder progress but does not break the game.

4. Entity Behavior:

- Enemies and dynamic entities react predictably based on programmed behaviors.
- Contact with certain entities (e.g., enemies or hazards) may cause damage, pushback, or restart from the last checkpoint.

5. Checkpoints and Lives:

- Levels are divided into checkpoints to save progress.
- Falling, failing puzzles, or taking damage may return the player to the last checkpoint rather than restarting the entire level.

IV.1.2 Implicit Rules

These are rules that are not directly stated but are learned through gameplay and observation:

1. Physics Consistency:

- Objects obey consistent physical rules, such as gravity, momentum, and collision response. Players can use this to predict and plan actions.

2. Timing and Momentum:

- Successful navigation often requires understanding the timing of moving platforms, swings, or obstacles.
- The player learns to anticipate entity movements and environmental reactions.

3. Environmental Interaction:

- Certain surfaces, slopes, or materials affect movement (e.g., slippery platforms or bouncy objects).
- Players intuitively learn which interactions are possible and safe.

4. Risk and Reward:

- Exploring hidden areas or manipulating objects may provide rewards (collectibles, shortcuts) but often introduces additional risk.
- Players learn to balance caution and experimentation.

5. Emergent Gameplay:

- Dynamic interactions between multiple objects and entities create unpredictable outcomes, teaching players that creative solutions are often required.

IV.2 Model of the game universe

The **game universe** of **Skystride** can be thought of as a **dynamic, physics-driven simulation** where all elements—player, entities, objects, and environment—interact according to defined rules and real-time physics. This model ensures that the world behaves consistently and predictably while still allowing emergent and creative gameplay.

IV.2.1 Core Components of the Universe

Player Character:

- The player is an active agent capable of movement, jumping, climbing, and interacting with the environment.
- The player's actions (e.g., pushing, jumping, landing) affect both dynamic objects and entities.

Dynamic Entities:

- NPCs, enemies, and other intelligent objects that react to player actions or environmental changes.
- They follow predefined AI behaviors but are influenced by physics (collisions, forces, momentum).

Interactive Objects:

- Platforms, levers, crates, bouncy surfaces, and other items that respond to physics and can affect gameplay.
- Objects can move, rotate, collapse, or combine with other objects depending on forces applied or player actions.

Environment:

- Terrain, walls, hazards, and level geometry that define the boundaries and challenges of the game world.
- The environment may include moving or destructible elements that are fully integrated into the physics simulation.

Physics Engine:

- Governs all movement, collisions, forces, and constraints in the game universe.
- Ensures consistent reactions for interactions between player, entities, objects, and the environment.

Game Logic System:

- Manages rules, objectives, checkpoints, and interactions between components.
- Determines when objectives are completed, when hazards trigger consequences, and how entities behave in response to player actions.

IV.2.2 Interaction Model

The game universe can be visualized as a **network of interdependent components**:

- **Player ↔ Objects:** Player can push, pull, ride, or otherwise manipulate objects; objects respond according to physical laws.
- **Player ↔ Entities:** Player may avoid, engage, or influence entities; entities react based on AI and physics.
- **Objects ↔ Objects:** Objects can collide, stack, fall, or move in response to other objects.
- **Entities ↔ Environment:** Entities navigate and interact with platforms, obstacles, and hazards in a realistic manner.
- **Player/Entities ↔ Physics Engine:** All movement, collisions, and forces are computed in real time to ensure realistic simulation.
- **Game Logic ↔ Universe:** Monitors and enforces rules, triggers events, and updates objectives based on interactions.

IV.2.3 Summary

The game universe is a **fully interactive simulation**, where all elements obey physics and respond dynamically to player and entity actions. This creates emergent gameplay, encourages exploration, and ensures that no two player experiences are exactly the same. The combination of AI-driven entities, physics-based objects, and reactive environments forms a cohesive, living world for the player to explore and master.

IV.3 Physics

The physical universe is designed to be **realistic, consistent, and responsive**, drawing inspiration from the physics systems used in classic 3D first-person games such as **Quake** and **Half-Life 1**. The game combines advanced physics simulation with dynamic entity interactions to create a world where every action has a tangible consequence.

IV.3.1 Rigid Body Dynamics (unimplemented)

All moving objects—including platforms, crates, and player characters—are modeled as rigid bodies with defined mass, velocity, and momentum. Collisions between objects are resolved using a physics engine that computes forces, impulses, and resulting responses, following principles similar to the implementations found in *Quake* and *Half-Life 1*.

IV.3.2 Gravity and Environmental Forces (implemented)

Gravity consistently influences all entities and objects, affecting movement, jumps, falls, and projectile trajectories. Environmental forces—including moving platforms, explosions, and player-applied interactions—impact objects in a realistic manner.

IV.3.3 Collision Detection and Response (implemented AABB system)

The game employs real-time collision detection to manage interactions between the player, dynamic entities, objects, and level geometry. Collisions trigger appropriate responses, including bouncing, sliding, stopping, or applying damage when necessary. Movement is further influenced by surface properties, such as friction on floors or slipperiness on ice.

Currently, collision detection is implemented using an Axis-Aligned Bounding Box (AABB) system, which simplifies calculations by enclosing objects within rectangular bounds aligned to the world axes. This approach efficiently handles most interactions while maintaining performance.

IV.3.4 Player Movement Physics (implemented)

The player can run, jump, and climb while momentum and inertia are taken into account, adhering to a physics-based movement model. Advanced mechanics—including momentum conservation during jumps, wall interactions, and object manipulation—enable precise, skill-based gameplay and movement mastery.

IV.3.5 Object Interaction (unimplemented)

Dynamic objects respond to player actions and interactions with other entities in accordance with physical laws. Objects can be pushed, lifted, stacked, or toppled, and their interactions may trigger chain reactions within the environment, creating emergent gameplay opportunities.

IV.4 Economy

The economy is **non-monetary** and does not involve a traditional currency system. Instead, the game focuses on **resource management** to support player survival and progression.

IV.4.1 Resource Collection

Players acquire essential resources, including ammunition, health packs, and temporary power-ups, which are necessary for overcoming enemies and navigating environmental challenges. These resources are strategically positioned throughout levels to encourage exploration, careful planning, and calculated risk-taking.



IV.4.2 Survival Mechanics

The deliberate scarcity of resources introduces a survival element, compelling players to manage and utilize items judiciously. Depletion of critical resources, such as ammunition, increases the difficulty of certain challenges and may necessitate alternative problem-solving strategies.

IV.4.3 Incentivized Exploration

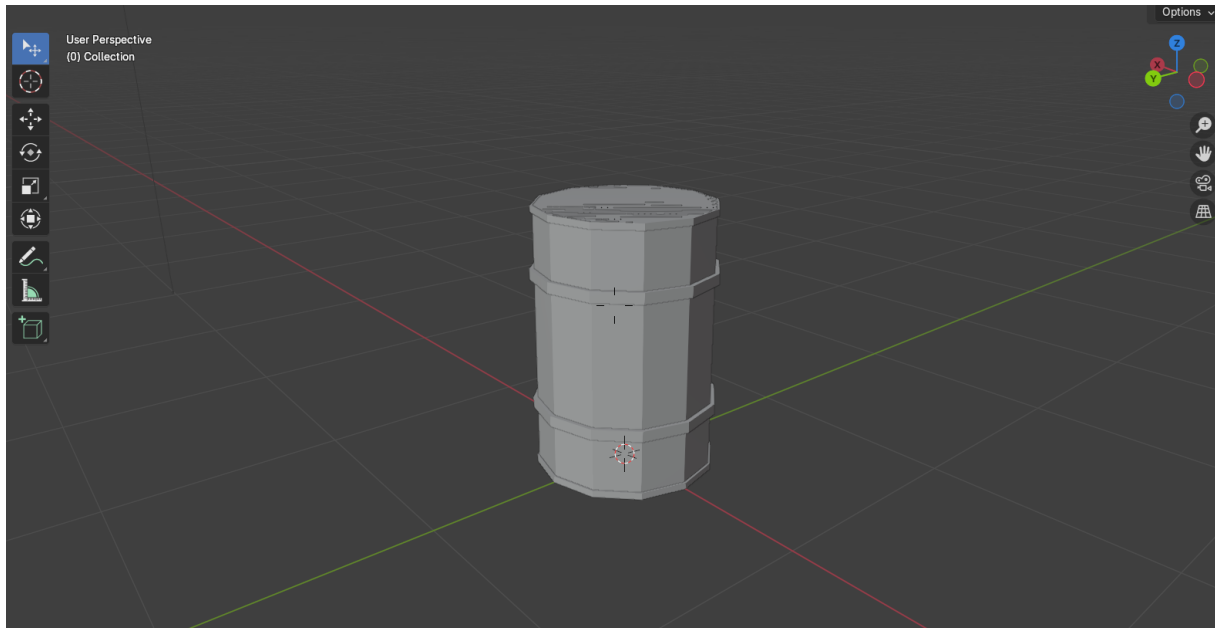
Hidden or difficult-to-reach areas frequently contain additional resources, incentivizing thorough exploration and interaction with the physics-driven environment. Acquiring these resources supports survival and can provide tactical advantages, such as extra ammunition prior to confronting challenging enemies.

IV.5 Objects

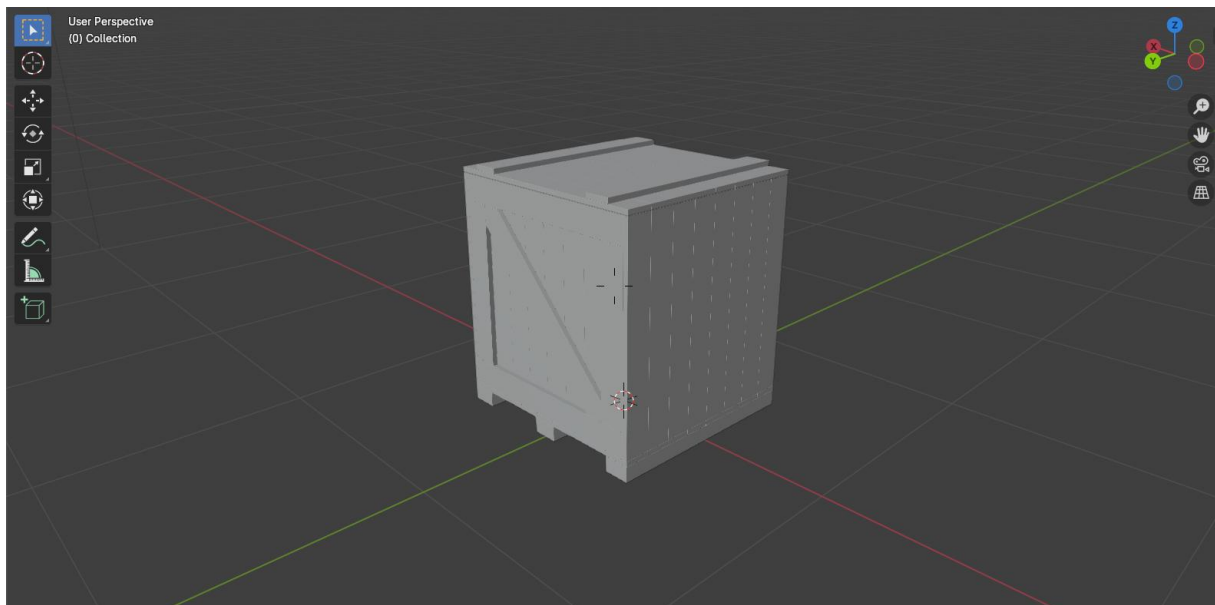
1. Picking Up and Moving Objects

Players can interact with designated environmental objects, such as crates, barrels, and movable platforms. Objects can be pushed, pulled, or lifted according to their weight and size, using physics-based mechanics. Certain objects may be employed strategically to solve puzzles, access elevated areas, or block hazards.

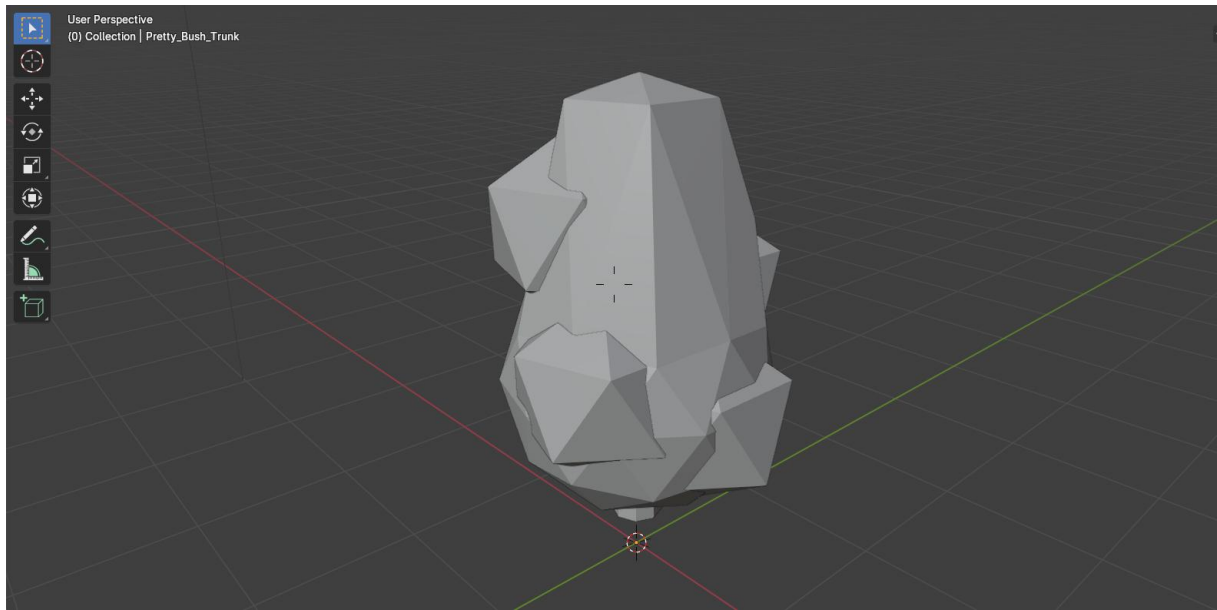
1. Barrel



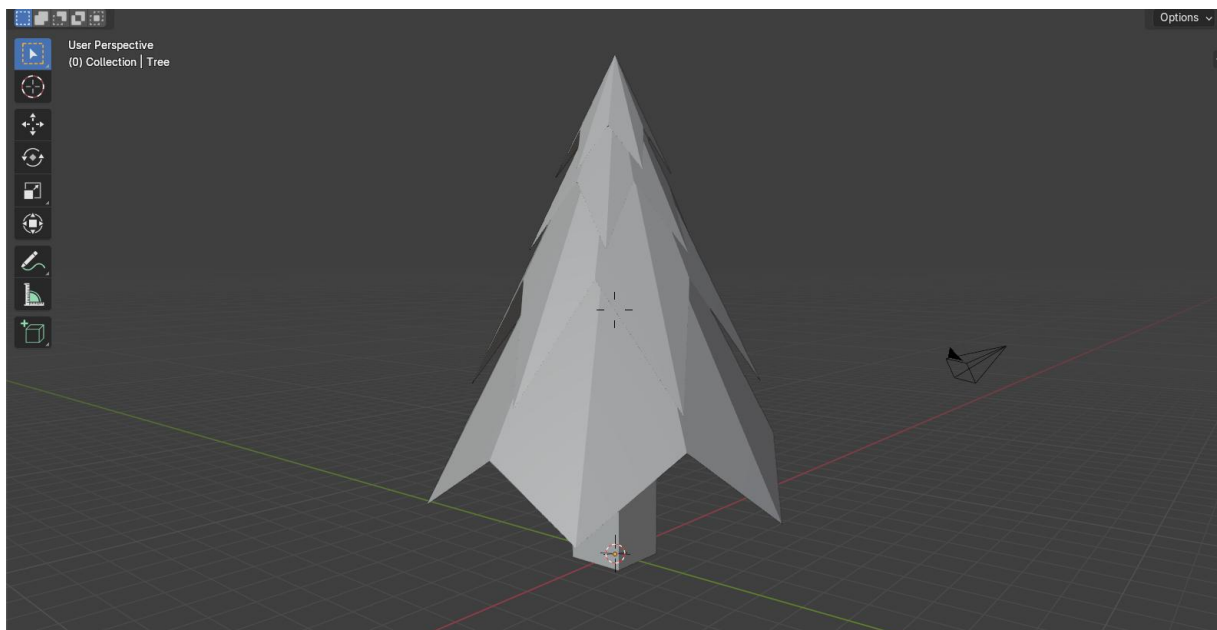
2. Box



3. Bush



4. Tree



2. Weapons and Combat Objects

Weapons are either spawned after defeating enemies or placed at predetermined locations within the level. Weapons have limited ammunition, encouraging strategic use and careful resource management. Each weapon interacts with the physics system, influencing projectile motion, recoil, and environmental impact.

1. Glock



2. Shotgun



3. Items and Resources

- **Ammunition:** Collected to replenish weapon supplies; can be found throughout levels or dropped by defeated enemies.
- **Medkits / Health Packs:** Restore player health; strategically placed to reward exploration or challenge players to reach them.
- **Special Items:** May provide temporary buffs, enable environmental interaction, or assist in puzzle-solving.

4. Enemy Drops

Defeated enemies may drop items such as ammunition, weapons, or health packs, depending on their type and strength. The rarity and value of drops are scaled to the challenge posed by the enemy, establishing a risk-reward system in which more formidable foes yield more valuable rewards.

V Story and Narrative

V.1 Current Narrative Status

At the current stage of development, the game does not feature a fully developed story or narrative. The focus is primarily on physics-based platforming, dynamic entity interactions, and skill-driven gameplay.

V.2 Current Narrative Context

- **Main Level:** The primary map available in the current version is *Vertigont*.
- **Objective:** Players must navigate the 3D environment, overcome dynamic obstacles, and defeat the main boss.
- **Gameplay-Driven Progression:** Rather than a story-driven experience, player progression is determined by level completion, resource management, and mastery of physics-based mechanics.

V.3 Potential for Future Narrative Integration

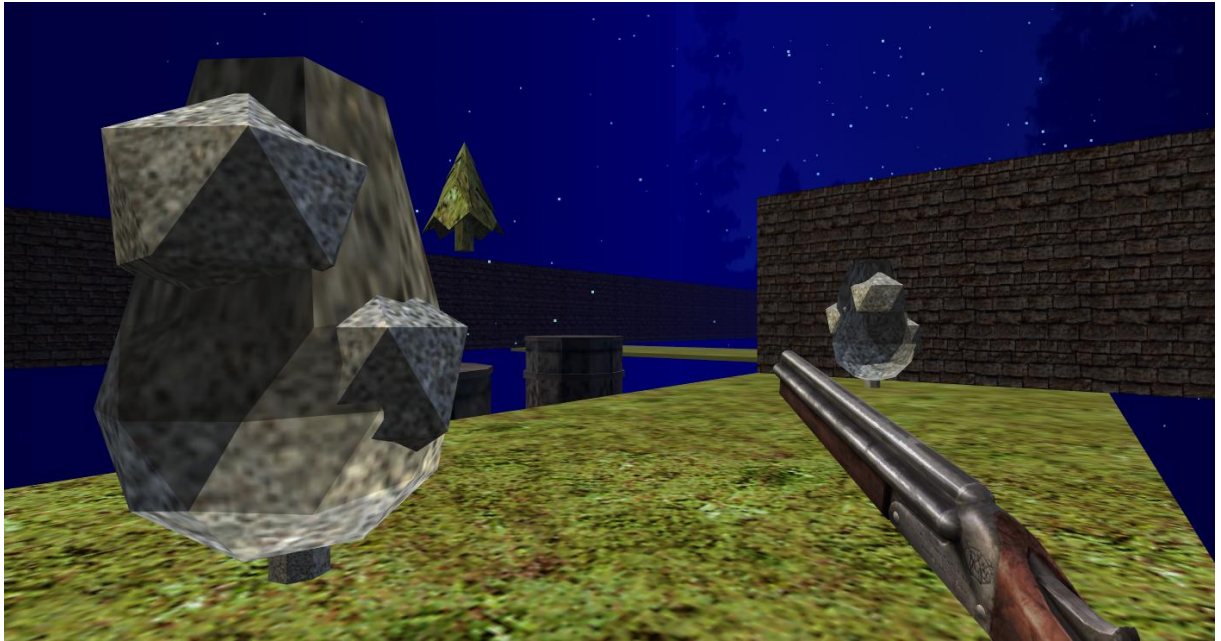
Although a complete story is not yet implemented, the game's framework supports the future inclusion of narrative elements, such as backstory, characters, or additional objectives beyond defeating the main boss. Features such as cutscenes, dialogue, or environmental storytelling could be incorporated in later updates to enhance the narrative experience while preserving the core gameplay mechanics.

VI Game World

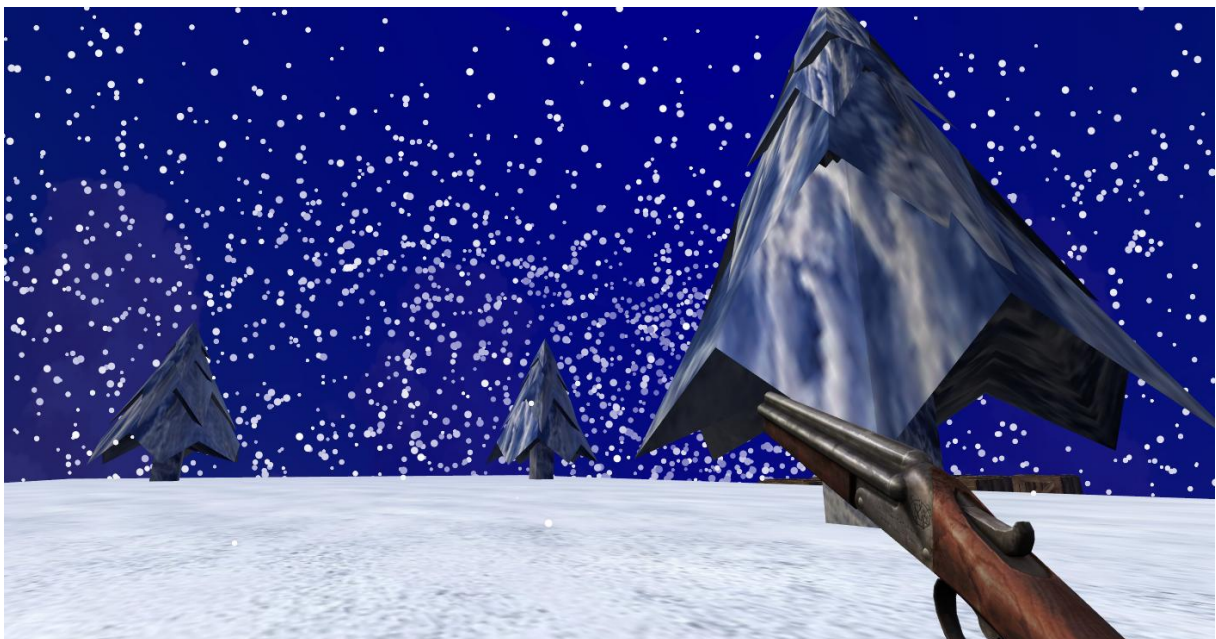
VI.1 Maps and Environments

The game features multiple maps, each with a unique visual theme and gameplay challenges:

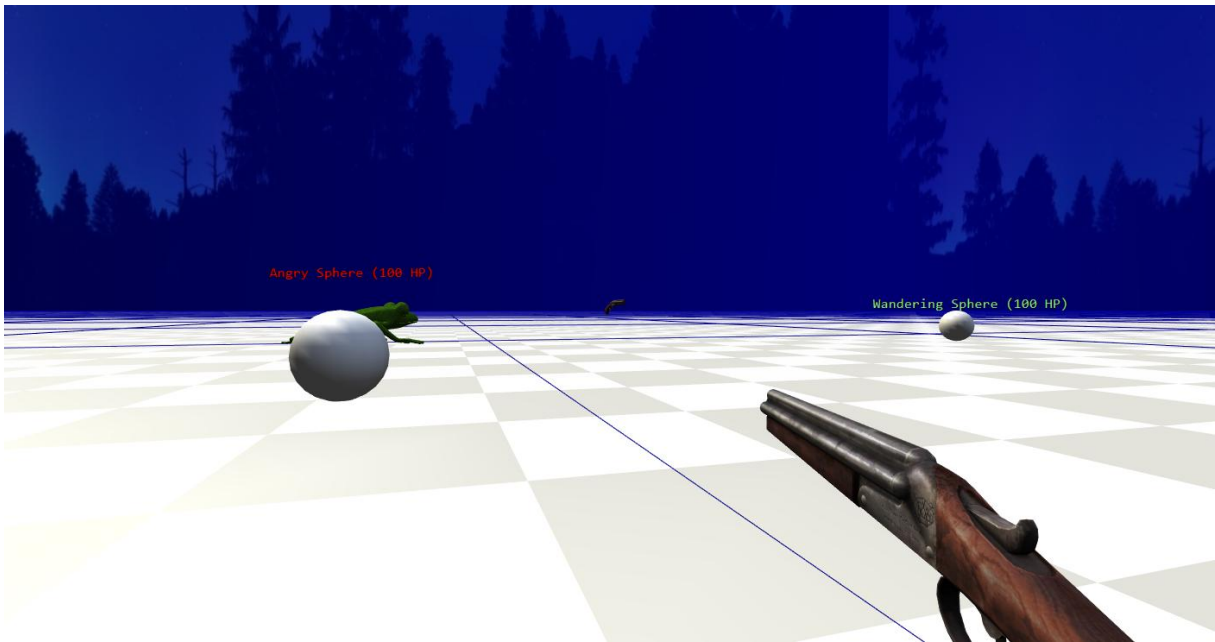
1. Forest - Lush, green environments with trees, foliage, and natural obstacles.



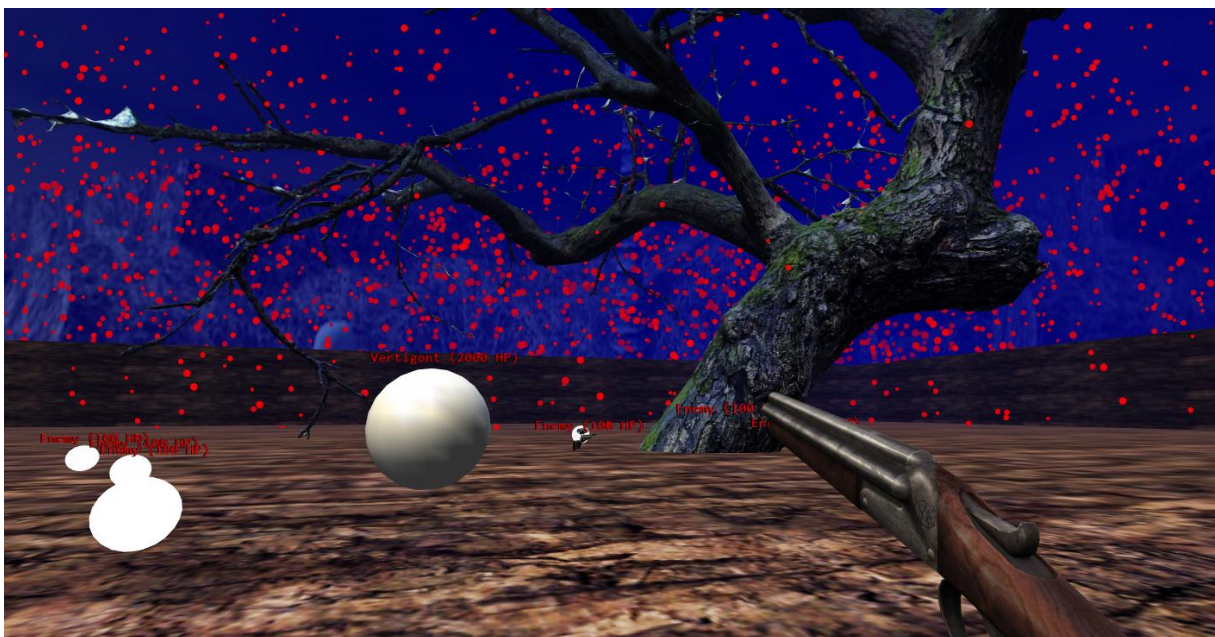
2. Arctic - Snow-covered terrain with icy surfaces, slippery platforms, and frozen hazards.



3. Template - modular, basic map designed for testing gameplay mechanics, physics interactions, and object behavior.



4. Vertigont - This is the main map in the current version of the game and includes the final boss encounter.



VII Interface

VII.1 HUD (Heads-Up Display)

The HUD provides essential gameplay information, including:

- Player health or hit points.
- Current ammunition or resources.
- Objective indicators or status messages, where applicable.

The HUD is designed to be minimalistic, reducing visual clutter and allowing players to focus on platforming and physics-based interactions. In developer mode, additional debug information may be displayed, such as coordinates, entity states, or physics details.

VII.2 Menus



A traditional menu system is not implemented in the current version of the game. Instead, the game features a console interface that allows players to:

- Change maps (e.g., switch between Forest, Arctic, Template, and Vertigont).
- Enter developer mode to access debugging tools, inspect physics, or test gameplay mechanics.
- Modify settings, spawn objects, or trigger special events within the game world.

The console provides a flexible and powerful alternative to standard menus, particularly during development and testing.

VII.3 Camera Model

The game employs a 3D first-person camera, offering an immersive perspective from the player character's viewpoint. Camera behavior is designed to:

- Follow the player's head movements and directional input smoothly.
- Allow free rotation and looking around via mouse or analog stick.
- Ensure clear visibility of interactive objects, platforms, and hazards.

The first-person perspective emphasizes precise movement, timing, and environmental awareness, complementing the physics-driven platforming mechanics.

VIII Audio, music, sound effects

An initial music track for the *Vertigont* map was generated using AI. This track remains incomplete and is currently not implemented in the game. No additional music tracks are included in the current version, leaving opportunities for future soundtracks to accompany different maps and gameplay events.

IX Game Art – intended style

IX.1 Visual Style and Inspiration

Half-Life 1 Influence:

The game adopts a stylized realism inspired by *Half-Life 1*, prioritizing clear shapes, functional environments, and easily recognizable objects. Environments are designed to be visually distinct, allowing players to immediately identify interactive elements, hazards, and navigable pathways.

Physics Integration:

Objects, platforms, and entities are visually designed to emphasize their physical properties, including weight, size, and potential movement. This allows players to anticipate interactions and leverage physics for problem-solving and platforming challenges.

Environmental Themes:

Maps such as Forest, Arctic, Template, and Vertigont each feature distinct visual identities while maintaining a coherent overall style. Textures, lighting, and geometry prioritize clarity, visibility, and gameplay readability rather than hyper-realistic detail.

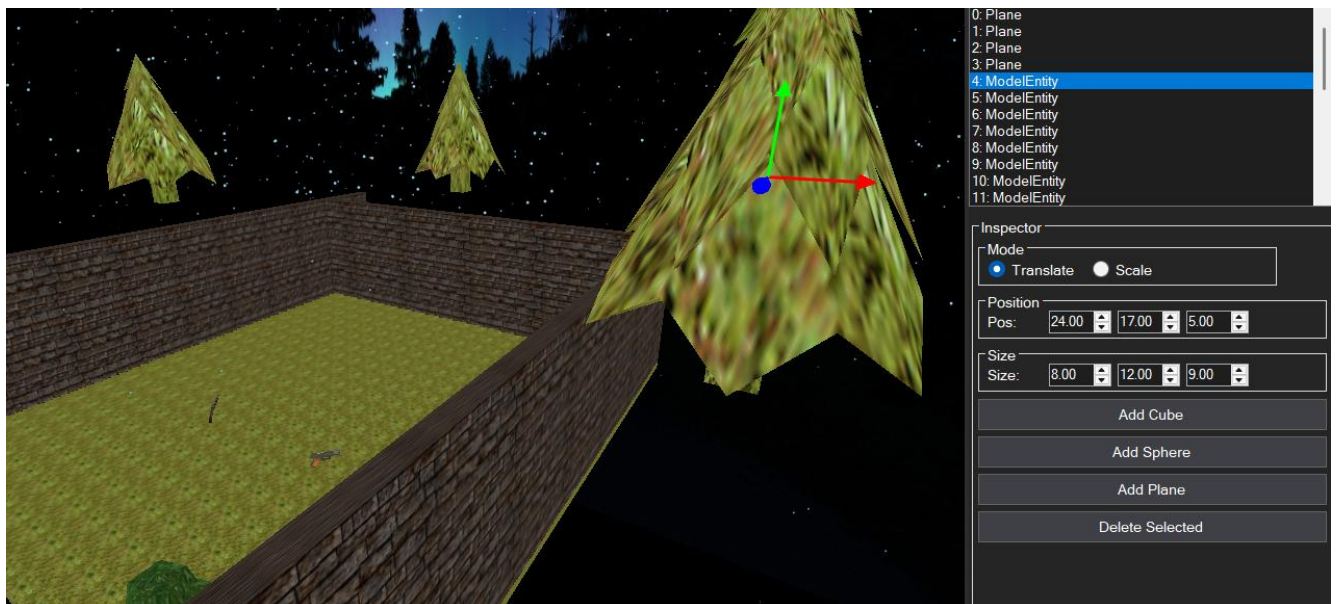
IX.2 Characters and Entities

Characters and enemies utilize simple yet expressive models, enabling players to quickly recognize behaviors and interaction patterns.

IX.3 Objects and Interactive Elements

Interactive objects—including crates, barrels, levers, and platforms—are designed to stand out visually from static environmental elements. Object models clearly communicate their function and physical properties, ensuring players understand how to interact with them.

X Map Editor



The Map Editor is designed to enable level designers and developers to efficiently create, modify, and test maps while maintaining full integration with the physics-driven game world.

X.1 Editor Overview

The editor provides a real-time editing environment where objects, platforms, hazards, and entities can be placed, moved, rotated, and scaled. All modifications are synchronized with the game engine, ensuring that physics behaviors, collisions, and interactions behave identically to actual gameplay, allowing designers to prototype new levels or modify existing ones efficiently.

X.2 Gizmo Controls

Objects within the editor are manipulated using visual gizmos, enabling:

- **Translation:** Move objects along the X, Y, or Z axes.
- **Rotation:** Rotate objects around any axis to adjust orientation.
- **Scaling:** Resize objects while preserving physical properties.

These controls provide intuitive, immediate feedback on object position and orientation relative to the environment.

X.3 Game Synchronization

Changes made in the editor are directly reflected in the game world:

- Dynamic objects immediately inherit physics properties, including mass, collisions, and momentum.
- Entities placed in the editor respond according to AI scripts and physics interactions.
- Map testing occurs in real time, without requiring export or game reload.

This ensures that levels created in the editor accurately match the player's in-game experience, reducing iteration time and minimizing errors.

X.4 Additional Features

- **Object Library:** Provides quick access to pre-made assets, including platforms, hazards, interactive objects, and enemies.
- **Snap-to-Grid and Alignment Tools:** Maintain consistent level design and platform alignment.
- **Developer Console Integration:** Allows map changes to be tested or debugged using console commands, including spawning objects, toggling physics properties, or activating developer mode.

XI Engine Systems

XI.1 Inventory

The player has an inventory system to manage collected weapons. Weapons can be switched using the 1–4 keys, allowing the player to quickly select their desired armament. When a weapon is picked up, any ammunition provided is automatically added to the corresponding weapon already in the player’s inventory, ensuring efficient resource management and seamless gameplay.

XI.2 Loading Screen

When a scene is loaded or a map is changed, a dedicated background thread handles object loading while a loading screen is displayed. This system progressively loads objects for the map in the background, optimizing scene transitions and reducing perceived wait times. By separating object loading from the main gameplay thread, the system ensures smoother performance and a more seamless player experience.

XI.3 Axis-Aligned Bounding Box Collision

The game utilizes an Axis-Aligned Bounding Box (AABB) collision system to manage interactions between the player, dynamic entities, objects, and level geometry. In this system, each object is enclosed within a rectangular bounding box aligned to the world axes (X, Y, Z). Collisions are detected by checking for overlaps between these boxes.

AABB collision provides a simple and efficient method for real-time collision detection, allowing the physics engine to calculate responses such as sliding, stopping, bouncing, or applying damage. While it does not account for object rotation within the bounding box, it is highly performant and well-suited to the physics-driven, platforming-focused gameplay of the current game.

XI.4 Sound Manager

The game features a dedicated Sound Manager responsible for loading and managing music tracks and sound effects for each scene. Upon scene or map loading, the Sound Manager ensures that all audio assets—such as background music, environmental sounds, and interactive object effects—are properly initialized and synchronized with gameplay. This system supports dynamic audio playback, allowing sounds to respond to in-game events while optimizing performance and memory usage during scene transitions.

XI.5 Non-Player Character (NPC)

The game includes a Non-Player Character (NPC) system to manage autonomous entities within the environment. NPCs interact with the player, dynamic objects, and environmental elements according to predefined behaviors and AI scripts.

Key features of the NPC system include:

- **Behavioral Logic:** NPCs follow scripted patterns or AI routines, responding to player actions and environmental changes.
- **Physics Integration:** NPCs are treated as physics-driven entities, responding realistically to collisions, forces, and interactions with movable objects.
- **Environmental Interaction:** NPCs can trigger or react to in-game events, such as moving platforms, hazards, or chain reactions caused by dynamic objects.
- **Combat and Resource Interaction:** NPCs may engage in combat with the player or interact with items and resources in the environment, contributing to emergent gameplay scenarios.

The NPC system is designed to support both predictable and emergent behaviors, enhancing the challenge, immersion, and dynamism of the game world.

XI.6 Distance Rendering System

The game employs a distance-based rendering system to optimize performance and maintain visual clarity across large environments. Objects, entities, and environmental elements are rendered based on their distance from the player, with details dynamically adjusted to balance performance and visual fidelity.

Key features include:

- **Culling:** Non-visible objects outside the camera's view or occluded by other geometry are excluded from rendering, further improving performance.
- **Seamless Transitions:** Changes in object detail occur smoothly to prevent noticeable visual popping or distraction.
- **Physics and Interactions:** Despite visual simplification at a distance, all objects maintain full physics properties, ensuring consistent gameplay interactions regardless of rendering detail.

This system allows large, complex levels to run efficiently while preserving the integrity of physics-driven interactions and gameplay mechanics.